Experiment 9

Student Name: Chetan Prakash **UID: 23BCS13776**
Branch: CSE **Section/Group: KRG 1A**
Semester: 6th **Date of Performance: 13/04/2026**
Subject Name: System Design **Subject Code: 23CSH-314**

1. **Aim**: To design and analyze a scalable Ticket Booking System (like BookMyShow/Ticketmaster) that allows users to discover events, lock seats in real-time, manage bookings, and perform payment transactions efficiently using distributed system architecture.

2. **Objective**:

- To identify functional and non-functional requirements of a ticket booking system
- To design High Level Architecture (HLD) for event discovery and booking platforms
- To understand real-time distributed seat locking mechanisms
- To design microservices for search, booking, seats, and payments
- To study transaction execution and double-booking prevention mechanisms
- To understand scalability challenges in high-traffic ticketing systems

3. **Tools:**
   - Draw.io / Excalidraw – System design diagrams
   - MySQL / PostgreSQL – User & transaction data
   - Redis – Caching & real-time data
   - Kafka – Event streaming (stock prices/orders)
   - Load Balancer – Traffic distribution
   - WebSockets – Real-time stock updates
   - Microservices Architecture • InfluxDB – Time-series stock data

4. System Requirements:

A. Functional Requirements:-

- Users should be able to search events by keyword, city, or date
- System should display detailed event info and interactive seat maps
- Users should be able to temporarily lock seats (TTL of 10 minutes)
  - Users should execute bookings and process payments
  - System should generate e-tickets (QR code + PDF)

3. Non-Functional Requirements:-

1) Scalability
a. Support 100 million daily active users (DAU)
b. Handle 95% Read / 5% Write traffic split
2) Consistency (CAP Theorem)
a. Booking Path: Consistency > Availability (CP Model)
b. Search Path: Availability > Consistency (AP Model)
3) Availability
a. Event discovery should maintain 99.99% Uptime SLA
4) Latency
a. Search and Event page loads < 200–300 ms
b. Booking execution < 500 ms (P99)
5) Reliability
a. Strict ACID transactions; no phantom reads or duplicate bookings

5. High Level Design (HLD):

System follows Microservices Architecture Main Components:

1. API Gateway
    a. Authentication (JWT)
    b. Authorization
    c. Routing
    d. Strict Rate Limiting (e.g., 10 req/min for seat locking)
2. User Service
    a. Registration & login
    b. User profile management
    c. User DB (PostgreSQL)
3. Search Service
    a. Queries Elasticsearch for rapid event discovery
    b. Uses Redis Cache for hot search results
4) Event & Seat Service
    a. Manages event metadata, venues, and shows
    b. Provides interactive seat maps (mapped to PostgreSQL)
5) Booking Service
    a. Acquires distributed locks via Redis Redlock
    b. Executes ACID transactions with optimistic locking

c. Updates seat status to 'BOOKED'

6) Payment Service

a. Integrates external gateways (Razorpay/Stripe)

b. Enforces idempotency to ignore duplicate attempts

7) Notification Service

a. Consumes Kafka messages asynchronously

b. Dispatches Email/SMS/Push alerts

8) Ticket Generation Service

a. Generates QR+PDF tickets and stores in S3

9) CDN (CloudFront)

a. Edge caching for posters and static assets

6. Booking Flow (Step-by-Step):

1) User hits API Gateway where JWT and rate limits are checked
2) User searches events and views the interactive seat map
3) User selects a seat to book
4) Booking Service acquires Redis Redlock (10-minute TTL)
5) PostgreSQL ACID transaction begins to lock seat
6) Payment Gateway is called to process transaction
7) Upon success, seat status updated to 'BOOKED'
8) Kafka publishes booking-events message
9) Notification Service sends Email/SMS alerts
10) Ticket Gen Service creates QR/PDF ticket in S3

7. Scalability Solution: To support 100 million DAU:

- Use Multi-Region Active-Active architecture with GeoDNS
- Use Redis for aggressive caching of search and event details
- Use CloudFront (CDN) to serve heavy static assets globally
- Use PostgreSQL read replicas to offload 95% browse traffic
- Use Elasticsearch for complex full-text and geo-spatial queries
- Use Kafka to decouple non-blocking tasks (notifications/analytics)
- Use Kubernetes (HPA/KEDA) for CPU and event-driven auto-scaling
- Use API Gateway for centralized security (WAF) and control

8. Learning Outcomes (What I Have Learnt):

- Learned how stock trading platforms are designed
- Understood real-time data streaming using Kafka
- Learned order validation and execution flow
- Understood portfolio management systems
- Learned importance of consistency in financial systems • Understood microservices and scalability
- Learned how exchanges (NSE/BSE) integrate with apps